

# SWIPEX

---

## **AI-Powered Swipe-Based Intelligent Job Discovery & Career Assistance Platform**

### **Project Report**

SUBMITTED BY

SHAIK SHAHIN

B.Tech Computer Science and Engineering

RISE Krishna Sai Prakasam Group Of Institutions

(Affiliated to Jawaharlal Nehru Technological University Kakinada)

Vallur::Ongole

Prakasam

## 1. Abstract

SwipeX is an AI-powered job discovery and career assistance platform designed to make job searching faster, simpler, and more personalized. Instead of relying only on traditional job-listing pages, the platform presents jobs through an interactive swipe-based experience. Users can review opportunities quickly, save relevant jobs, and apply to suitable positions.

The platform also provides intelligent career assistance through resume analysis, ATS-oriented scoring, recommendation workflows, application tracking, and notifications. The backend exposes REST APIs and uses PostgreSQL for persistent data storage. The production architecture separates the frontend and backend, with the frontend deployed on Render, the FastAPI backend deployed on Render, and PostgreSQL hosted through Render.

## 2. Introduction

Job seekers commonly face a large number of listings, repetitive searches, unclear job suitability, and difficulty understanding whether a resume matches a role. Recruiters also need a structured way to publish jobs and manage applicants.

SwipeX addresses these issues by combining an intuitive job-discovery interface with data-driven assistance. The system organizes job information, evaluates resume relevance, supports applications, and provides timely notifications in one platform.

## 3. Problem Statement

- Traditional job portals can require repeated searching and manual comparison of many listings.
- Job seekers may not know whether their skills and resume match a particular job.
- Important job and application updates can be missed without timely notifications.
- Recruiters need a simple workflow for posting jobs and managing applicants.
- Career platforms often separate job discovery, resume analysis, and application tracking.

## 4. Objectives

- Provide an interactive swipe-based job discovery experience.
- Recommend jobs according to user information and job relevance.
- Allow users to save jobs and submit applications.
- Analyze resumes and provide ATS-oriented scoring and history.
- Provide notifications for relevant jobs and application events.
- Provide recruiter workflows for job posting and applicant management.
- Maintain reliable relational data using PostgreSQL.
- Deploy the platform using a scalable frontend/backend architecture.

## 5. Proposed System

The proposed system is a web-based platform with separate frontend and backend layers. The frontend provides the user interface for job discovery, authentication, saved jobs, applications, resume features, and recruiter workflows. The FastAPI backend handles authentication, business logic, recommendation-related operations, resume processing, job management, applications, notifications, and database communication.

### Core User Flow

- Register or log in.
- Complete or maintain the user profile.
- Browse jobs through the discovery interface.
- Swipe or interact with jobs and save suitable opportunities.
- Apply to selected jobs.
- Use resume/ATS analysis to understand resume suitability.
- Receive notifications and track application activity.

## 6. Major Modules

### 6.1 Authentication and User Management

The platform supports user registration and login. User records are stored in PostgreSQL and are associated with application, saved-job, notification, and resume-analysis information.

### 6.2 Job Discovery

Jobs are displayed through a user-friendly discovery workflow. Job information includes details such as title, description, location, skills, and recruiter information. The swipe-oriented interface reduces the effort required to review multiple opportunities.

### 6.3 Saved Jobs

Users can save jobs for later review. Saved jobs create a direct relationship between a user and a job and allow relevant opportunities to be retained without immediately applying.

### 6.4 Applications

The application module records job applications and connects each application to the corresponding user and job. This provides a structured application history.

### 6.5 Resume and ATS Analysis

Resume-related functionality evaluates resume suitability and stores score history. This allows users to understand their resume performance and observe score-related information over time.

### 6.6 Notifications

Notifications provide timely information about jobs and application-related events. The notification system is connected to users and, where applicable, specific jobs.

### 6.7 Recruiter Features

Recruiters can manage job postings and applicants through dedicated backend workflows.

## 7. Technology Stack

| Layer           | Technology   | Purpose                                     |
|-----------------|--------------|---------------------------------------------|
| Frontend        | React JS     | User interface and client-side interactions |
| Backend         | Django       | REST API and business logic                 |
| ORM             | SQLAlchemy   | Database models and persistence             |
| Database        | PostgreSQL   | Persistent relational data                  |
| Deployment      | Render       | Frontend hosting                            |
| Deployment      | Render       | Backend hosting                             |
| Version Control | Git / GitHub | Source-code management                      |

## 8. System Architecture

SwipeX follows a three-layer production architecture:

- Browser / Client — provides the interactive web interface.
- Render Frontend — hosts the production JavaScript frontend.
- Render Django Backend — exposes APIs and executes application logic.
- PostgreSQL — stores users, jobs, applications, notifications, saved jobs, and resume-score history.

The frontend communicates with the backend through HTTP APIs. The backend communicates with PostgreSQL through SQLAlchemy. This separation keeps presentation, business logic, and persistent storage independently manageable.

## 9. Database Design

The production database is organized around six core tables. Foreign-key relationships maintain consistency between users, jobs, applications, notifications, saved jobs, and resume analysis history.

| Table                | Purpose                        | Important Relationship                                                      |
|----------------------|--------------------------------|-----------------------------------------------------------------------------|
| users                | Stores platform users          | Referenced by jobs, applications, notifications, saved jobs, resume history |
| jobs                 | Stores job postings            | posted_by references users                                                  |
| applications         | Stores applications            | user_id → users; job_id → jobs                                              |
| notifications        | Stores user/job notifications  | user_id → users; job_id → jobs                                              |
| resume_score_history | Stores resume scoring history  | user_id → users                                                             |
| saved_jobs           | Stores saved job relationships | user_id → users; job_id → jobs                                              |

### Database Integrity

The migrated production dataset contained 22 users, 19 jobs, 13 applications, 300 notifications, 12 resume-score-history records, and 4 saved-job records. Foreign-key integrity checks reported zero broken references.

## 10. Backend and API Design

The FastAPI backend provides endpoints for authentication, jobs, saved jobs, applications, resume/ATS features, and recruiter operations. APIs act as the communication layer between the frontend and PostgreSQL-backed application services.

## 11. Frontend Design

The frontend is designed around a clean job-discovery workflow. The main user experience focuses on quickly understanding a job and deciding whether to retain or pursue it.

### Main Frontend Areas

- Authentication — registration and login.
- Job discovery — browsing and swipe-oriented interaction.
- Saved jobs — reviewing retained opportunities.
- Applications — viewing application activity.
- Resume/ATS — accessing resume scoring and history.
- Notifications — viewing relevant updates.
- Recruiter interface — job and applicant management.

## 12. Deployment

### Production Architecture

Browser → Render Frontend → Django Render Backend → Render PostgreSQL

### Production Deployment

- Frontend production deployment: <https://swipex-1.onrender.com>
- Backend production deployment: <https://swipex-backend-production.up.render.app>
- Frontend routing was configured for clean job-page navigation.
- Database migration was completed on Render PostgreSQL.
- Secrets and credentials are kept outside the source repository.

The deployment separates the presentation layer from API services and database storage, making the system easier to maintain and update.

## 13. Testing and Validation

Testing was performed across deployment, authentication, routing, database migration, and data integrity.

| Test Area                         | Result              |
|-----------------------------------|---------------------|
| Backend production deployment     | Successful          |
| Production registration           | Successful          |
| Production login                  | Successful          |
| Jobs HTML availability            | Successful          |
| Clean /jobs route                 | Successful          |
| /jobs-page compatibility redirect | Successful          |
| database migration                | Successful          |
| Foreign-key integrity             | 0 broken references |

## 14. Project Data Migration

The migration process moved project data from the local environment into the production PostgreSQL database while preserving relationships.

- Back up the local database before migration.
- Preserve existing production users where required.
- Insert users and jobs while creating ID mappings.
- Translate foreign keys for applications, notifications, saved jobs, and resume history.
- Correct PostgreSQL sequences after insertion.
- Run foreign-key and integrity checks after migration.

## 15. Future Enhancements

- Improve recommendation quality using richer user and job signals.
- Expand resume parsing and automated skill extraction.
- Add recruiter analytics and candidate ranking.
- Introduce automated email and push notifications.
- Improve accessibility and responsive behavior.
- Add CI/CD automation for safer releases.
- Introduce application monitoring and observability.
- Provide advanced search, filtering, and recommendation explanations.

## 16. Conclusion

SwipeX demonstrates a complete AI-assisted career platform that combines job discovery with practical career-support features. Its swipe-based interaction provides a simple way to review opportunities, while saved jobs, applications, resume analysis, recommendations, and notifications create a connected workflow.

The system uses a modern web architecture with React JS, Django, SQLAlchemy, PostgreSQL and Render. The successful production deployment and migration validation show that the platform is structured beyond a basic prototype and can serve as a foundation for further development.

## 17. Repository and Live Deployment

### Live Application

Frontend: <https://swipex-ai.vercel.app>

Backend: <https://swipex-backend-production.up.railway.app>

### Repository

The project source code is maintained through Git and GitHub. The repository contains the application source and deployment configuration while credentials and secrets are excluded from version control.

### Production Status

- Frontend deployed on Render.
- Backend deployed on Render.
- PostgreSQL database hosted through Render.
- Production authentication validated.
- Production job routing validated.
- Database migration completed.
- Foreign-key integrity verified with zero broken references.

## Project Summary

SwipeX is an AI-powered, swipe-based job discovery and career assistance platform built to simplify the path from discovering a job to understanding suitability, saving an opportunity, applying, and receiving updates. The project integrates frontend development, backend API engineering, relational database design, deployment, data migration, and AI-assisted career functionality in a single application.

-----END-----